

Lab 10 – Solutions

December 2, 2024

1 Quadrature formulas

The definite integral

$$I(f) = \int_a^b f(x)dx$$

can be computed by **replacing** f with an approximation f_n . Using polynomial interpolation, we get **quadrature formula** of the form

$$I_n(f) = \sum_{j=0}^n \alpha_j f(x_j)$$

where x_j are the so called quadrature **nodes** and α_j the related **weights**.

A quadrature formula is said to have **degree of exactness** p if it exactly integrates the polynomial of degree $\leq p$.

A formula with $n + 1$ nodes has at most degree of exactness equal to $2n + 1$ (hint: think about the Gaussian quadrature rule, characterized by the maximum possible degree of exactness).

Midpoint quadrature rule

$$I_0(f) = (b - a) f\left(\frac{a + b}{2}\right).$$

$$\text{If } f \in \mathcal{C}^2([a, b]), \quad E_0(f) = \frac{(b - a)^3}{24} f''(\xi), \quad \xi \in (a, b).$$

Trapezoidal quadrature rule

$$I_1(f) = \frac{b - a}{2} [f(a) + f(b)].$$

$$\text{If } f \in \mathcal{C}^2([a, b]), \quad E_1(f) = -\frac{(b - a)^3}{12} f''(\xi), \quad \xi \in (a, b).$$

Simpson quadrature rule

$$I_2(f) = \frac{b - a}{6} \left[f(a) + 4f\left(\frac{a + b}{2}\right) + f(b) \right].$$

$$\text{If } f \in \mathcal{C}^4([a, b]), \quad E_2(f) = -\frac{(b - a)^5}{2880} f^{(4)}(\xi), \quad \xi \in (a, b).$$

Similarly to the case of polynomial interpolation we can divide the integration interval into **subintervals** and apply on each of them a quadrature formula with a **low number** of nodes. In this way the generic quadrature formula becomes

$$I_{n,m}(f) = \sum_{k=0}^{m-1} \sum_{j=0}^n \alpha_j^{(k)} f(x_j^{(k)})$$

with $x_j^{(k)}$ and $\alpha_j^{(k)}$ the quadrature nodes and weights on the k^{th} subinterval respectively. The order of accuracy of a composite quadrature formula coincides with the rate of the convergence of the error.

Composite midpoint rule

$$I_{0,m}(f) = h \sum_{k=0}^{m-1} f(\bar{x}_k), \quad h = \frac{b-a}{m}.$$

$$\text{If } f \in \mathcal{C}^2([a, b]), \quad E_{0,m}(f) = \frac{(b-a)}{24} h^2 f''(\xi), \quad \xi \in (a, b).$$

Composite trapezoidal rule

$$I_{1,m}(f) = \frac{h}{2} \sum_{k=0}^{m-1} (f(x_k) + f(x_{k+1})) = \frac{h}{2} [f(x_0) + f(x_m)] + \sum_{k=1}^{m-1} f(x_k), \quad h = \frac{b-a}{m}.$$

$$\text{If } f \in \mathcal{C}^2([a, b]), \quad E_{1,m}(f) = -\frac{b-a}{12} h^2 f''(\xi), \quad \xi \in (a, b).$$

Composite Simpson rule

$$I_{2,m}(f) = \frac{h}{6} \left[f(x_0) + 2 \sum_{r=1}^{m-1} f(x_{2r}) + 4 \sum_{s=0}^{m-1} f(x_{2s+1}) + f(x_{2m}) \right], \quad h = \frac{b-a}{m}.$$

$$\text{If } f \in \mathcal{C}^4([a, b]), \quad E_{2,m}(f) = -\frac{b-a}{2880} h^4 f^{(4)}(\xi), \quad \xi \in (a, b).$$

Exercise 10.1.

Consider the following integrals

$$I_1 = \int_0^1 x^3 dx = \frac{1}{4} \quad \text{and} \quad I_2 = \int_0^1 x^5 dx = \frac{1}{6}.$$

- Implement the simple midpoint, trapezoidal and Simpson rule in three different MATLAB functions.
- Using the code at the previous question, implement composite version of the midpoint, trapezoidal and Simpson rule.
- Compute the integral I_1 using the midpoint rule with 1 and 10 nodes and the composite trapezoidal rule with 2 and 10 nodes. Comment on the obtained results.
- Compute I_1 and I_2 with the composite Simpson rule with 3 and 7 nodes. Comment on the results.
- Estimate the number of nodes needed to compute I_2 with a tolerance of 10^{-3} using the composite trapezoidal and Simpson rule.
- Verify the convergence order of the three composite methods.

Solution Exercise 10.1.

ex_10_1.m

```

%% Exercise 10.1
clc;clear all;close all;
%% a) Implement simple midpoint,
% trapezoidal and Simpson rule in MATLAB functions
%% b) Implement composite midpoint,
% trapezoidal and Simpson rule in MATLAB functions
%% c)

f1 = @(x) x.^3;
f2 = @(x) x.^5;
a = 0; b = 1;

% Midpoint rule: the number of subintervals is equal to the number of total nodes.

I1_m_1 = composite_midpoint(f1, a, b, 1);
I1_m_10 = composite_midpoint(f1, a, b, 10);

% Check
fprintf("Composite MidPoint Rule\n");
fprintf("True Value | m=1 | m=10 \t\n");
fprintf(" %4g\t %4g %4g\n",1/4, I1_m_1, I1_m_10);
%%

% The function to be integrated is a polynomial of order 3. Hence the rule,
% of degree 1, is not able to compute the integral exactly.
% The error decreases while increasing the number of nodes
% (i.e., increasing the number of subintervals)

% Trapezoidal rule: the number of subintervals is equal to the number
% of total nodes minus 1.

I1_t_2 = composite_trapezoidal(f1, a, b, 1);
I1_t_10 = composite_trapezoidal(f1, a, b, 9);
% Check
fprintf("Composite Trapezoidal Rule\n");
fprintf("True Value | m=2 | m=10 \t\n");
fprintf(" %4g\t %4g %4g\n",1/4, I1_t_2, I1_t_10);

% Same behaviour as before, since the trapezoidal rule is of degree 1.

% d)

% Simpson rule: the number of subintervals is equal to (n - 1)/2,
% where n is the number of total nodes.

I1_s_3 = composite_simpson(f1, a, b, 1);
I1_s_7 = composite_simpson(f1, a, b, 3);
% Check
fprintf("Composite Simpson Rule\n");
fprintf("True Value | m=3 | m=7 \t\n");
fprintf(" %4g\t %4g %4g\n",1/4, I1_s_3, I1_s_7);

% The result does not depend on the number of nodes.
% The reason of this is the fact that Simpson rule has degree of exactness
% equal to 3 and the integral on any subinterval is computed exactly!

I2_s_3 = composite_simpson(f2, a, b, 1);
I2_s_7 = composite_simpson(f2, a, b, 3);
% Check
fprintf("Composite Simpson Rule\n");
fprintf("True Value | m=1 | m=10 \t\n");
fprintf(" %4g %4g %4g\n",1/6, I2_s_3, I2_s_7);
% Instead, when considering the approximation of I_2,
% the integrand function is a polynomial of order higher than 3,
% hence the rule is not exact.

% e)

% Recall that the error for the composite trapezoidal rule is given in
% slide 6.

```

```

% Really you should have to compute the inf norm the the derivative
% term in the error estimate; however in this case you can perform
% this computation by hand: how?

h_star = sqrt(12 * 1e-3 / ((b-a) * 20));
m_star = ceil((b-a) / h_star);

fprintf("Estimated number of nodes m (Trapezoidal): %d\n", m_star);
% ... and the number of nodes is m_star + 1.

% Check
I1_t_star = composite_trapezoidal(f1, a, b, m_star);
% it should be smaller than 10^-3
fprintf("Tolerance | Error \n");
err_star = abs(1/4 - I1_t_star);
fprintf(" %g %g\n", 1e-3, err_star);

% Similarly for the composite Simpson rule:

h_star = ((180 * 1e-3) / ((b-a) * 120))^(1/4);
m_star = ceil((b-a) / (2*h_star));
fprintf("Estimated number of nodes m (Simpson): %d\n", m_star);

% ... and the number of nodes is 2 m_star + 1.
fprintf("Tolerance | Error \n");
err_star = abs(1/6 - I2_s_7);
fprintf(" %g %g\n", 1e-3, err_star);

% f)
for i = 0:9
    m = 2^i;
    integr_m(i+1) = composite_midpoint(f2, a, b, m);
    integr_t(i+1) = composite_trapezoidal(f2, a, b, m);
    integr_s(i+1) = composite_simpson(f2, a, b, m);
end

err_m = abs(1/6 - integr_m);
err_t = abs(1/6 - integr_t);
err_s = abs(1/6 - integr_s);

p_m = -diff(log(err_m)) / log(2);
p_t = -diff(log(err_t)) / log(2);
p_s = -diff(log(err_s)) / log(2);
fprintf("Estimated order of accuracy MidPoint: %d\n", p_m(end));
fprintf("Estimated order of accuracy Trapezoidal: %d\n", p_t(end));
fprintf("Estimated order of accuracy Simpson: %d\n", p_s(end));

% As expected, the error decreases quadratically in h for both midpoint
% and trapezoidal rules. For Simpson rule instead, convergence is of fourth order in h.

% Graphical visualization of the order of accuracy.
h = 1./2.^[0:9]; % [1/1; 1/2; 1/4; ...; 1/2^9]
figure
loglog(h, err_m, 'rs-', 'LineWidth', 2)
hold on, box on
loglog(h, err_t, 'bs-', 'LineWidth', 2)
loglog(h, err_s, 'ms-', 'LineWidth', 2)
loglog(h, h.^2, 'g--', 'LineWidth', 2)
loglog(h, h.^4, 'g-', 'LineWidth', 2)
set(gca, 'FontSize', 16)
set(gca, 'LineWidth', 1.5)
xlabel('h', 'FontSize', 16)
ylabel('error', 'FontSize', 16)
leg = legend('Midpoint', 'Trapezoidal', 'Simpson', 'O(h^2)', 'O(h^4)', 'Location', 'southeast'
);
set(leg, 'FontSize', 16)
set(leg, 'LineWidth', 1.5)

```

midpoint.m

```
function int = midpoint(f, a, b)
% MIDPOINT Computes the integral of a function f between a and b using
% the midpoint quadrature rule.
%
% Inputs : f = function handle of the function to be integrated
%          a = left limit of the interpolation interval
%          b = right limit of the interpolation interval
% Outputs : int = integral approximation

int = (b-a)*f( (a+b)/2 );

end
```

trapezoidal.m

```
function int = trapezoidal(f, a, b)
% TRAPEZOIDAL Computes the integral of a function f between a and b using
% the trapezoidal quadrature rule.
%
% Inputs : f = function handle of the function to be integrated
%          a = left limit of the interpolation interval
%          b = right limit of the interpolation interval
% Outputs : int = integral approximation

int = (b-a)/2*(f(a)+f(b));

end
```

simpson.m

```
function int = simpson(f, a, b)
% SIMPSON Computes the integral of a function f between a and b using
% the simpson quadrature rule.
%
% Inputs : f = function handle of the function to be integrated
%          a = left limit of the interpolation interval
%          b = right limit of the interpolation interval
% Outputs : int = integral approximation

int = (b-a)/6*(f(a) + 4*f((a+b)/2) + f(b));

end
```

composite_midpoint.m

```
function int = composite_midpoint(f, a, b, m)
% COMPOSITE_MIDPOINT Computes the integral of a function f between a and b using
% composite midpoint quadrature rule on m subintervals.
%
% Inputs : f = function handle of the function to be integrated
%          a = left limit of the interpolation interval
%          b = right limit of the interpolation interval
%          m = number of subintervals
% Outputs : int = integral approximation

x = linspace(a, b, m+1);
int = 0;

for (i = 1:m)
    int = int + midpoint(f, x(i), x(i+1));
end

end
```

composite_trapezoidal.m

```

function int = composite_trapezoidal(f, a, b, m)
    % COMPOSITE_TRAPEZOIDAL Computes the integral of a function f between a and b using
    % composite trapezoidal quadrature rule on m subintervals.
    %
    % Inputs : f = function handle of the function to be integrated
    %          a = left limit of the interpolation interval
    %          b = right limit of the interpolation interval
    %          m = number of subintervals
    % Outputs : int = integral approximation

    x = linspace(a, b, m+1);
    int = 0;

    for (i = 1:m)
        int = int + trapezoidal(f, x(i), x(i+1));
    end

end

```

composite_simpson.m

```

function int = composite_simpson(f, a, b, m)
    % COMPOSITE_SIMPSON Computes the integral of a function f between a and b using
    % composite simpson quadrature rule on m subintervals.
    %
    % Inputs : f = function handle of the function to be integrated
    %          a = left limit of the interpolation interval
    %          b = right limit of the interpolation interval
    %          m = number of subintervals
    % Outputs : int = integral approximation

    x = linspace(a, b, m+1);
    int = 0;

    for (i = 1:m)
        int = int + simpson(f, x(i), x(i+1));
    end

end

```

Exercise 10.2. Consider the integral

$$I = \int_{-1}^1 f(x) dx$$

and the following quadrature formulas

$$Q_1 = \frac{2}{3} \left[2f\left(-\frac{1}{2}\right) - f(0) + 2f\left(\frac{1}{2}\right) \right],$$

$$Q_2 = \frac{1}{4} \left[f(-1) + 3f\left(-\frac{1}{3}\right) + 3f\left(\frac{1}{3}\right) + f(1) \right].$$

- Which is the degree of exactness of these formulas? Justify the answer.
- Use the quadrature formulas above to approximate the integral

$$\int_1^3 \log(x) dx.$$

Which is the best one giving in term of accuracy ?

Solution Exercise 10.2.

ex_10_2.m

```
%% Exercise 10.2
clc; clear all; close all

% a)

weights_1 = 2/3*[2 -1 2];
nodes_1 = [-1/2 0 1/2];
f = @(x) x.^0; sum(weights_1 .* f(nodes_1)) %expected 2
f = @(x) x.^1; sum(weights_1 .* f(nodes_1)) %expected 0
f = @(x) x.^2; sum(weights_1 .* f(nodes_1)) %expected 1/3
f = @(x) x.^3; sum(weights_1 .* f(nodes_1)) %expected 0
f = @(x) x.^4; sum(weights_1 .* f(nodes_1)) % inexact, expected 2/5

weights_2 = 1/4*[1 3 3 1];
nodes_2 = [-1 -1/3 1/3 1];
f = @(x) x.^0; sum(weights_2 .* f(nodes_2)) %expected 2
f = @(x) x.^1; sum(weights_2 .* f(nodes_2)) %expected 0
f = @(x) x.^2; sum(weights_2 .* f(nodes_2)) %expected 1/3
f = @(x) x.^3; sum(weights_2 .* f(nodes_2)) %expected 0
f = @(x) x.^4; sum(weights_2 .* f(nodes_2)) % inexact, expected 2/5

% Both formulae are exact up to degree 3.

% b)

% The integral can be rewritten as
% int_{-1}^{1} log(t+2) dt;
% we then apply the quadrature rule.

f = @(t) log(t+2)
Q1 = sum(weights_1 .* f(nodes_1));
Q2 = sum(weights_2 .* f(nodes_2));

F = @(x) x.*log(x) - x;
int_exact = F(3) - F(1);

err_Q1 = abs(int_exact - Q1)
err_Q2 = abs(int_exact - Q2)

% The second rule behaves better.
```

Exercise 10.3. Consider the following integral

$$I = \int_{-1}^1 \sqrt{1-x^2} dx = \frac{\pi}{2}.$$

- Compute I using the composite trapezoidal and Simpson rules with a number of nodes $n = 3^k$, $k = 1, \dots, 9$.
- Estimate the order of accuracy of the two methods and comment on the results.

Solution Exercise 10.3.

ex_10_3.m

```
%% Exercise 10.2
clc;clear all;close all

a = -1; b = 1;
f = @(x) sqrt(1 - x.^2);

for k = 1:9
    n(k) = 3^k;
    m_t(k) = n(k) - 1;
    m_s(k) = (n(k) - 1) / 2;
    int_t(k) = composite_trapezoidal(f, a, b, m_t(k));
    int_s(k) = composite_simpson(f, a, b, m_s(k));
end
[n' int_t' int_s']

% With these results we see that both the methods converge to pi/2.

% b)

err_t = abs(pi/2 - int_t);
err_s = abs(pi/2 - int_s);

p_t = -diff(log(err_t)) / log(3);
p_s = -diff(log(err_s)) / log(3);

% The rate of convergence of the two methods is 1.5:
% the reduction of the order of accuracy is due to the fact that the
% integrand function is not sufficiently smooth to apply the previous
% convergence results
% (C^2 regularity for the trapezoidal rule, C^4 regularity for the Simpson rule,
% but the function is only C^0).
```

Exercise 10.4 (Exam June 17, 2024). Consider the composite Simpson's rule for approximating the integral.

$$I(f) = \int_a^b f(x) dx$$

Exam June 17, 2024

- Using the provided MATLAB function which implements the composite Simpson's rule, approximate the following integral:

$$I(f) = \int_a^b \frac{x(x^3 + 7x + 2) + 6}{(x^2 + 3)^2} dx$$

and compute the approximate integral $I_n(f)$ using $n = 4, 8, 16, 32$ equally spaced intervals. Display the results in the box with all available significant digits (use the MATLAB command `format long e`), along with the MATLAB instructions used.

- Knowing that $I(f) = \frac{5}{6}$, calculate the error $E_n(f) = |I(f) - I_n(f)|$ using $n = 4, 8, 16, 32$ equally spaced intervals. Display the results along with the MATLAB instructions used.

3. Comment on the results obtained in part b in light of the theoretical findings.
4. Plot the error as a function of h to graphically verify the behavior predicted by theory. Generate a figure with the plot and upload it.
5. Repeat part a using the function $f(x) = -3x^2 + 7x$. Display the results in the box along with the MATLAB instructions used, and comment on the result in light of the theoretical expectations.

Solution Exercise 10.4.

ex_10_4.m

```
%% Exercise 10.4
clc; clear; close all;

%% 1
f = @( x ) ( x .* ( x.^ 3 + 7* x + 2 ) + 6 ) ./ ( x.^ 2 + 3 ) .^ 2 ;
format long e
a =0; b=1; M=4;
I_m_4 = composite_simpson( f, a, b, M )
M=8;
I_m_8 = composite_simpson( f, a, b, M )
M=16;
I_m_16 = composite_simpson( f, a, b, M )
M=32;
I_m_32 = composite_simpson( f, a, b, M )
%% 2
Iex = 5/6;
Iappr =[ I_m_4 ; I_m_8 ; I_m_16 ; I_m_32 ] ;
err = abs( Iex *ones(4 ,1) - Iappr)

%% 3
%According to theory, the error in the composite Simpson's rule is proportional
% to h^4. Consistent with this prediction, doubling the number of subdivisions,
% and thus halving their size, reduces the error by a factor of 2^4 = 16,
% as shown by the following ratios:
format default
err(1)/err(2)
err(2)/err(3)
err(3)/err(4)

%% 4
n = [4 8 16 32] ;
h = (b-a)./n ;
figure
loglog(h, err, 'g', 'LineWidth',2)
hold on
loglog(h, 1.1*h.^4/h(1)^4*err(1),'r--', 'LineWidth',2)
legend ( 'err', 'h^4')

%% 5
f = @(x) -3*x.^2 + 7*x;
format long e
a =0; b=1; M=4;
I_m_4 = composite_simpson( f, a, b, M )
M=8;
I_m_8 = composite_simpson( f, a, b, M )
M=16;
I_m_16 = composite_simpson( f, a, b, M )
M=32;
I_m_32 = composite_simpson( f, a, b, M )
% It is easily verified that the exact value of the integral is 5/2
% Therefore, the code returns the exact result, apart from truncation errors,
% regardless of the number of subdivisions used. This is consistent with
% the theoretical result that Simpson's rule has an accuracy of degree 3,
% meaning it integrates polynomials of degree less than or equal to 3
% exactly, such as the one proposed in this exercise.
```

Exercise 10.5 (Exam June 27, 2023). Consider the composite trapezoidal rule for approximating the integral.

$$I(f) = \int_a^b f(x) dx$$

Exam June 27, 2023

1. Using the provided MATLAB function which implements the composite trapezoidal rule, approximate the following integral:

$$I(f) = \int_a^b 3 \sin\left(\frac{\pi}{3}x\right) - \exp(-3x) dx$$

and compute the approximate integral $I_n(f)$ using $n = 8, 16, 32, 64$ equally spaced intervals. Display the results in the box along with the MATLAB instructions used.

2. Knowing that $I(f) = \frac{9}{2\pi} \frac{e^{-3}-1}{3}$, calculate the error $E_n(f) = |I(f) - I_n(f)|$ using $n = 8, 16, 32, 64$ equally spaced intervals. Display the results along with the MATLAB instructions used.
3. Comment on the results obtained in part b in light of the theoretical findings.
4. Plot the error as a function of h to graphically verify the behavior predicted by theory. Generate a figure with the plot and upload it.
5. Repeat part a using the function $f(x) = -3x + 7$. Display the results in the box along with the MATLAB instructions used, and comment on the result in light of the theoretical expectations.

Solution Exercise 10.5.

ex_10_5.m

```
%% Exercise 10.5
clc; clear; close all;

%% 1
f = @( x ) 3*sin(pi*x/3) -exp(-3*x);
a =0; b=1; M=8;
I_m_8 = composite_trapezoidal( f, a, b, M )
M=16;
I_m_16 = composite_trapezoidal( f, a, b, M )
M=32;
I_m_32 = composite_trapezoidal( f, a, b, M )
M=64;
I_m_64 = composite_trapezoidal( f, a, b, M )

%% 2
Iex = 9/(2*pi) + (exp(-3)-1)/3;
Iappr=[I_m_8; I_m_16; I_m_32; I_m_64];
err = abs(Iex*ones(4,1)-Iappr)

%% 3
% Analyzing the errors found in the previous step, it is observed that
% by doubling n(i.e., halving h = 2/n), the error becomes approximately
% a quarter of its previous value.
% This is in line with the theory, which states that the error satisfies
% |I - I_n| = O(h^2).

%% 4
% A possible way to generate the requested graph is to plot the errors
% as a function of h = (b - a)/n along with the behavior of h^2.
% The MATLAB commands are:
n = [8 16 32 64];
h = (b-a)./n;
figure()
```

```

loglog(h, err, 'g', 'LineWidth', 2)
hold on
loglog(h,h.^2,'r--','LineWidth', 2)
legend('err','h2', 'Location','northwest')

%% 5
f = @(x) -3*x + 7;
a =0; b=1; M=8;
I_m_8 = composite_trapezoidal( f, a, b, M )
M=16;
I_m_16 = composite_trapezoidal( f, a, b, M )
M=32;
I_m_32 = composite_trapezoidal( f, a, b, M )
M=64;
I_m_64 = composite_trapezoidal( f, a, b, M )

```